Banco de Dados

Dependência Funcional, Multivalorada, Transitiva

Antes da normalização...

Já sabemos transformar um **DER em tabelas**.

Mas surge uma pergunta:

> **Como saber se nossas tabelas estão bem organizadas?**

Para responder isso, precisamos entender as **dependências entre os dados**.

O que significa "depender"?

CPF	Nome
123	Ana
456	João
789	Maria

Se eu souber o **CPF**, consigo descobrir o **nome** da pessoa.

> O **nome depende do CPF**.

Dependência funcional

Acontece quando:

> **Um atributo determina o valor de outro atributo.**

Representamos assim:

`CPF → Nome`

Lemos: **CPF determina Nome**, ou **Nome depende funcionalmente de CPF**.

O que significa "determinar"?

> Para um mesmo valor de A, só pode existir **um único valor de B**.

```
CPF = 123  →  Nome = Ana     (sempre)
```

Não pode acontecer:

```
123 → Ana
123 → João
```

Exemplo: aluno

matrícula	nome	curso
101	Ana	Informática
102	João	Redes
103	Maria	Informática

```
matricula → nome
matricula → curso
```

A matrícula identifica o aluno, ou seja, ela **determina** seus dados.

E o contrário?

Será que podemos afirmar `nome → matricula`?

Não necessariamente.

matrícula	nome
101	Ana
102	João
103	Ana

Duas pessoas podem ter o mesmo nome.

> A dependência funcional tem **direção**.

Como identificar uma dependência funcional

Pergunte:

> **Sabendo o valor de A, consigo descobrir um único valor de B?**

- Se **sim** → `A → B`
- Se **pode haver mais de um valor** → não é dependência funcional

Cuidado: analise o **significado** dos dados, não apenas as linhas que aparecem na tabela. Uma dependência funcional é uma regra que **vale sempre**, não uma observação que vale só para as poucas linhas que você tem na frente.

Uma regra importante: a chave primária

A chave primária normalmente **determina os demais atributos** da tabela.

```
ALUNO(
    matricula PK,
    nome,
    curso
)
```

```
matricula → nome
matricula → curso
```

Outro exemplo

```
PRODUTO(
    cod_produto PK,
    nome,
    preco
)
```

```
cod_produto → nome
cod_produto → preco
```

Cada produto possui **um** nome e **um** preço armazenado.

Cuidado!

Em `ALUNO(matricula PK, nome, curso)` temos:

```
matricula → nome
matricula → curso
```

Mas isso **não** significa:

```
curso → matricula
```

Um curso possui **vários** alunos.

Exercício rápido

cod_filme	título	ano
1	Matrix	1999
2	Titanic	1997
3	Matrix Reloaded	2003

Qual dependência funcional podemos afirmar?

A) `cod_filme → título`

B) `título → cod_filme`

C) `ano → cod_filme`

D) `título → ano`

Resposta

A) `cod_filme → título`

O código identifica o filme. Também vale `cod_filme → ano`.

Não são dependências funcionais:

`ano → cod_filme`: vários filmes podem ter o mesmo ano.
`título → cod_filme`: títulos podem se repetir (remakes, por exemplo).

E quando a chave possui vários atributos?

```
MATRICULA(
    matricula,
    cod_disciplina,
    nota
)
```

Chave: **(matricula, cod_disciplina)**

Por quê? Porque um aluno pode cursar várias disciplinas, e uma disciplina tem vários alunos.

Dependência da chave composta

matrícula	disciplina	nota
101	BD	8
101	Web	9
102	BD	7

```
(matricula, disciplina) → nota
```

Precisamos dos **dois** valores para saber qual é a nota.

Dependência parcial

> Um atributo depende de **apenas uma parte** da chave composta.

```
MATRICULA(
    matricula,
    cod_disciplina,
    nome_aluno,
    nome_disciplina,
    nota
)
```

Chave: **(matricula, cod_disciplina)**

Dependência parcial

Depende da chave **completa**:

```
(matricula, cod_disciplina) → nota
```

Dependem de **parte** da chave (são **dependências parciais**):

```
matricula → nome_aluno
cod_disciplina → nome_disciplina
```

Por que isso é um problema?

matrícula	disciplina	nome_aluno
101	BD	Ana
101	Web	Ana
101	Algoritmos	Ana

O nome "Ana" fica repetido. Se ele mudar, precisamos alterar várias linhas.

Isso gera **repetição**, **inconsistência** e **anomalias de atualização**.

Dependência transitiva

```
ALUNO(
    matricula PK,
    nome,
    cod_curso,
    nome_curso
)
```

```
matricula → cod_curso
cod_curso → nome_curso
```

Logo: `matricula → nome_curso` (de forma **indireta**).

Dependência transitiva

Acontece quando:

```
A → B
B → C
logo, A → C
```

```
matricula
     ↓
cod_curso
     ↓
nome_curso
```

O nome do curso não depende **diretamente** da matrícula.

Por que isso é um problema?

matrícula	nome	cod_curso	nome_curso
101	Ana	10	Informática
102	João	10	Informática
103	Maria	20	Redes

Se o nome de um curso mudar, precisamos alterar várias linhas.

Como resolver?

Separando as informações:

```
ALUNO(matricula PK, nome, cod_curso FK)

CURSO(cod_curso PK, nome_curso)
```

```
ALUNO
   │ cod_curso
   ↓
CURSO
```

Cada informação fica no lugar adequado.

Dependência multivalorada

Imagine um aluno que pode ter:

- vários **telefones**;
- vários **e-mails**.

E essas informações são **independentes entre si**.

```
Ana
  Telefones: 9999-1111, 9888-2222
  E-mails:   ana@gmail.com, ana@ifce.edu.br
```

O problema

aluno	telefone	email
Ana	9999-1111	ana@gmail.com
Ana	9999-1111	ana@ifce.edu.br
Ana	9888-2222	ana@gmail.com
Ana	9888-2222	ana@ifce.edu.br

Criamos combinações desnecessárias: os telefones não têm relação com os e-mails.

Dependência multivalorada

```
aluno ↠ telefone
aluno ↠ email
```

O símbolo `↠` representa uma **dependência multivalorada**.

> Um registro pode possuir **vários valores independentes** de determinado atributo.

Como resolver?

```
ALUNO(matricula PK, nome)

TELEFONE_ALUNO(matricula FK, telefone)

EMAIL_ALUNO(matricula FK, email)
```

Agora cada conjunto de valores é armazenado separadamente.

Comparando (1/2)

Funcional

```
CPF → nome
```

Um valor determina um único valor.

Parcial

```
(matricula, disciplina) → nota
matricula → nome_aluno
```

Um atributo depende de **parte** da chave composta.

Comparando (2/2)

Transitiva

```
matricula → cod_curso
cod_curso → nome_curso
```

A dependência acontece **por meio de** outro atributo.

Multivalorada

```
aluno →→ telefone
```

Um registro possui **vários valores independentes**.

Resumindo

Tipo	Ideia principal
Funcional	A determina B
Parcial	B depende de parte da chave
Transitiva	A determina C através de B
Multivalorada	A possui vários valores independentes

Uma dica para lembrar 🧠

FUNCIONAL — "Um determina o outro." → `CPF → Nome`

PARCIAL — "Só uma parte da chave determina." → `matricula → nome_aluno`

TRANSITIVA — "Vai passando pelo meio." → `A → B → C`

MULTIVALORADA — "Um pode ter vários." → `Aluno →→ Telefone`

Exercício final

```
PEDIDO(
    cod_pedido,
    cod_produto,
    nome_produto,
    quantidade,
    cod_cliente,
    nome_cliente
)
```

Chave: **(cod_pedido, cod_produto)**

Identifique: uma dependência **total**, uma **parcial** e uma **transitiva**.

Resposta

```
(cod_pedido, cod_produto) → quantidade
```

✅ Total — depende da chave completa.

```
cod_produto → nome_produto
```

❌ Parcial — depende de parte da chave.

```
cod_pedido → cod_cliente → nome_cliente
```

❌ Transitiva — chega ao nome do cliente de forma indireta.

Por que estamos estudando isso?

Porque agora vamos começar a estudar a **NORMALIZAÇÃO**.

Ela utiliza essas ideias para:

- reduzir redundâncias;
- evitar inconsistências;
- diminuir anomalias;
- organizar melhor as tabelas.

Próximo passo

1ª Forma Normal — 1FN

E a primeira pergunta será:

> O que significa dizer que uma tabela está na **1FN**?